

Messenger interno do Portal do Cliente

Estudo de viabilidade & plano técnico de implementação

Objetivo: central de atendimento própria (estilo WhatsApp para o cliente) para o pós-venda, reduzindo a dependência de WhatsApp API + Kommo.

Data: 14 de junho de 2026 · **Projeto:** cbc-contratos (gerador de contratos + portal)

Veredito: Viável — com ressalvas · **Esforço:** 7 a 11 semanas

Análise: mapeamento do código + arquitetura + modelagem de custo (Supabase Pro / Netlify)

Sumário

- 0 Sumário executivo
- 1 Objetivo e escopo
- 2 O que já existe (reaproveitamento)
- 3 Arquitetura proposta
- 4 Segurança e LGPD
- 5 O que se perde tirando o Kommo
- 6 Custo (3 cenários)
- 7 Pré-requisitos (Fase 0)
- 8 Plano de construção em fases
- 9 Riscos e mitigações
- 10 Recomendação final
- A Anexos técnicos

o · Sumário executivo

Viável — com ressalvas

Dá para construir um messenger próprio excelente, e o terreno já está mais pronto do que parece: cerca de **60% da fundação** (login do cliente por link, app instalável, notificação push, banco de dados e design) já existe e está em produção. O que falta é o chat em si — mensagens ao vivo, central de atendimento e mídia. **Mas trocar 100% do WhatsApp não é seguro no curto prazo**, por uma limitação real do iPhone com notificações (seção 3.5).

~60%

da base já pronta
(login, app, push, banco)

7–11
sem.

para substituir o
pós-venda com confiança

~R\$ 0

custo extra/mês no
início (100 atend./dia)

12

clientes ativos hoje —
janela ideal para começar

As três verdades que sustentam este estudo

1. **Não é um fim de semana** — são **~2 meses de trabalho focado** para fazer direito (com segurança e testes).
2. **O iPhone + notificação push é a barreira real.** Sem uma rede de segurança (SMS/e-mail), uma fatia relevante dos clientes (especialmente iOS) não recebe aviso e volta ao WhatsApp.
3. **O Kommo não era só chat:** ele roda a régua de cobrança automática e o funil. Tirar sem reescrever isso é risco direto de receita.

I · Objetivo e escopo

Criar um **messenger próprio embutido no portal do cliente**, com duas faces:

- **Para o cliente:** interface estilo WhatsApp, instalável como app no celular (PWA). Texto, áudio, foto, vídeo e documentos.
- **Para o escritório:** central de atendimento robusta e 100% customizada (caixa de entrada multi-atendente).

Meta de negócio: após o cliente fechar contrato, concentrar a comunicação do pós-venda nesse canal próprio, reduzindo a dependência de WhatsApp API + Kommo.

Features obrigatórias

- Envio e recebimento de **documentos**.
- **Gravação de áudio** e **ouvir áudios** em até 2x.
- Envio e recebimento de **fotos e vídeos**.

Fora de escopo

Chamadas e videochamadas — permanecem no Google Meet agendado.

Princípios permanentes (do escritório)

Reduzir a ansiedade do cliente · liberar tempo do escritório · zero trabalho manual recorrente · design elegante · sem modo escuro · sem barra dourada fixa no topo.

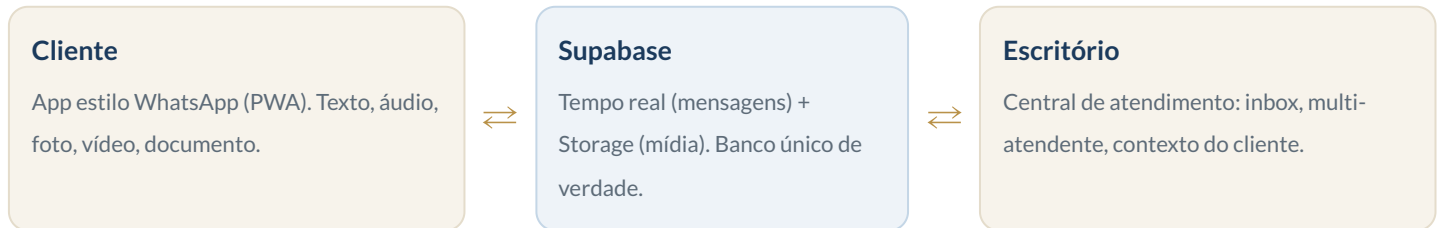
2 · O que já existe (reaproveitamento)

O ponto de partida é favorável: você não começa do zero. O portal foi lançado em 11/06/2026 e ainda tem só **12 clientes ativos** — a janela perfeita para introduzir o chat sem precisar migrar volume legado.

Já existe e está em produção	Como o messenger reaproveita
Login do cliente por link (tabela <code>cliente_portal_tokens</code> , 1.011 links ativos)	É o login do messenger. Zero trabalho novo de autenticação; 1 link ativo por cliente.
App instalável (PWA) — manifesto dinâmico + service worker + cache offline	O chat herda. Pequenos ajustes (atalho, ícones, badge).
Notificação push (web-push/VAPID já configurado, tabela <code>portal_push_subs</code>)	Avisa "nova mensagem" reaproveitando 100% da tubulação. Hoje com 0 assinaturas (nunca exercitado).
Supabase Pro (banco usando só 2,5% — 202 MB de 8 GB)	Mensagens de texto cabem folgadoíssimo. Storage e tempo real no mesmo projeto.
Tempo real (Supabase Realtime) já usado no painel admin (<code>App.jsx</code>)	Mesmo padrão (<code>postgres_changes</code>) entrega o inbox e o chat ao vivo.
Feature "Dúvidas" do portal (<code>portal_perguntas</code>)	É o esqueleto/predecessor do chat — cada dúvida vira a 1ª mensagem de uma conversa.
Espelhos ADVBOX/Asaas (processo, fase, financeiro)	Alimentam a coluna de contexto do atendente, sem nova integração.
Design do portal (creme/navy/dourado, mobile-first, sem dark mode)	O chat herda o visual elegante já exigido no brief.
Motor de respostas do bot (<code>buildLawsuitAnswer</code> , traduz "juridiquês")	Vira a resposta rápida "colar status do processo" na central.

3 · Arquitetura proposta

Princípio central de custo e desempenho: **o que trafega pesado (mídia e mensagens ao vivo) não passa pelo servidor da Netlify** — vai direto entre o navegador e o Supabase. A Netlify só processa o "control-plane" (validar token, gravar metadados, assinar URLs, disparar push).



3.1 Modelo de dados

4 tabelas principais (conversas, mensagens, participantes/leitura) + 1 bucket privado de mídia. **1 conversa por cliente** (estilo WhatsApp), amarrada ao `advbox_customer_id` que já vem do token. Apenas a tabela de mensagens e a de conversas entram no tempo real. **Mídia nunca no banco** — só a referência (caminho, tipo, tamanho, duração).

```
-- conversa = 1 por cliente (estilo WhatsApp), não por processo
create table msg_conversas (
  id          bigint generated always as identity primary key,
  advbox_customer_id bigint not null,
  token       text references cliente_portal_tokens(token),
  status      text not null default 'aberta',          -- aberta|resolvida|arquivada
  atribuido_a uuid references auth.users(id),          -- atendente dono (multi-atendente)
  etiquetas   text[] default '{}',
  ultima_msg_em timestampz,
  ultima_previa text,                                -- preview p/ inbox
  nao_lidas_escritorio int default 0,
  nao_lidas_cliente   int default 0,
  unique (advbox_customer_id)                        -- 1 conversa por cliente
);

create table msg_mensagens (
  id          bigint generated always as identity primary key,
  conversa_id bigint not null references msg_conversas(id) on delete cascade,
  direcao     text not null,                          -- in (cliente) | out (escritório)
  autor_id    uuid references auth.users(id),          -- "quem respondeu" (multi-atendente)
  tipo        text not null default 'texto',          -- texto|audio|foto|video|doc
  texto       text,
  midia_path  text, midia_mime text, midia_bytes bigint, midia_dur_s int, thumb_path text,
  client_msg_id uuid,                                -- idempotência (evita duplicar no retry)
  lido_em     timestampz,
  criado_em   timestampz default now()
);
create unique index on msg_mensagens (conversa_id, client_msg_id) where client_msg_id is not null;

alter publication supabase_realtime add table msg_mensagens, msg_conversas;
```

3.2 Entrega em tempo real

Decisão: **Supabase Realtime** (modo `postgres_changes`, que lê o próprio registro gravado) como canal primário, com **polling leve (20–30s)** só como reserva quando a conexão cair. Nunca usar polling pela Netlify como mecanismo primário — estouraria o número de chamadas.

- **Cliente:** assina só a própria conversa. O conteúdo sai por função protegida por token; pelo tempo real trafega apenas o "sinal" de que há algo novo.
- **Atendente:** assina o inbox inteiro (atualiza lista e contadores ao vivo) e a conversa aberta.
- **Efêmeros** ("digitando", "online") via canais que não gravam no banco.

3.3 Mídia (documentos, áudio em 2x, foto, vídeo)

Armazenamento no **Supabase Storage** (bucket privado), com upload e download **direto navegador↔armazenamento** via URL assinada (a Netlify só emite a URL). Todas as 4 features são viáveis:

Feature	Status	Como
Documentos	Viável	Anexo vai direto ao bucket; no banco fica só a referência. Abre por link assinado (expira em 1h).
Áudio + ouvir em 2x	Viável	Gravação nativa do navegador (<code>MediaRecorder</code>); player com 1x / 1,5x / 2x preservando o tom da voz. iPhone grava em mp4/aac e Android em webm/opus — detectado automaticamente.
Fotos	Viável	Miniatura gerada na hora (transformação de imagem nativa do Supabase). Foto comprimida no navegador antes de subir (corta custo).
Vídeos	Viável	1º quadro vira a capa; limite de tamanho/duração imposto (ex.: 50 MB / 120s).

Bloqueador a corrigir antes (pré-requisito)

Hoje o site **bloqueia microfone e câmera globalmente** (arquivo `client/public/_headers : Permissions-Policy: microphone=(), camera=()`). Sem liberar isso para a página do portal, a gravação de áudio/foto/vídeo **falha em silêncio**. Correção simples, porém obrigatória.

3.4 Fluxo de envio de mídia (bytes nunca passam pelo servidor)

1. Navegador pede uma **URL de upload assinada** (função valida o token, o tipo e o tamanho).
2. Navegador **envia o arquivo direto** para o armazenamento.
3. Navegador **confirma a mensagem** (a função grava só a referência no banco).
4. Tempo real + push avisam o outro lado. No download, uma URL assinada curta valida a posse.

3.5 Identidade, app instalável e o ponto crítico: notificações

A identidade reaproveita o token já existente (sem senha, sem cadastro), com endurecimento: dar validade ao token, tirá-lo da URL (guardar no aparelho) e adicionar campo de e-mail para o fallback.

O ponto que decide se "o cliente só fala por aqui" se sustenta é a **notificação**. A verdade, sem maquiagem:

Plataforma	Realidade do aviso (push)
Android (~62% da base)	Fácil. Funciona com o app aberto ou fechado, sem nem precisar instalar. Único senão: economia de bateria pode atrasar.
iPhone (~33% da base)	Limitação real e intransponível. O aviso só funciona se o cliente adicionar o app à Tela de Início (iOS 16.4+). No Safari normal não existe push. Taxa realista de instalação no iPhone: ~25% — ou seja, a maioria dos clientes de iPhone não receberia aviso nenhum.

Solução obrigatória: escada de notificação

Como sempre haverá clientes sem push, o aviso precisa ter camadas que se cobrem:

1. **Push imediato** quando há assinatura válida (cobre Android).
2. **Ao vivo dentro do app** + número vermelho no ícone, se estiver aberto.
3. **Rede de segurança: SMS e/ou e-mail** com link direto de volta, disparado automaticamente se o cliente não ler em ~15 minutos.

Dependência nova: o SMS/e-mail é um serviço externo que *o projeto não tem hoje* (ex.: Zenvia/Twilio/AWS, ~R\$ 0,05–0,15 por SMS) e não há coluna de e-mail no cadastro (o celular já existe no espelho de clientes). Volume estimado: **30 a 80 mensagens de fallback por dia**. O texto sempre puxa o cliente de volta ao messenger, nunca pede para responder no WhatsApp.

3.6 Central de atendimento (lado escritório)

Nova seção "**Atendimento**" dentro do painel admin já existente, com layout clássico de central de mensagens em 3 painéis:

Painel	Conteúdo
A • Caixa de entrada	Filtros (Minhas / Não atribuídas / Todas / Resolvidas), busca por nome/CPF/processo/texto, contador de não-lidas, faixa dourada nas "minhas conversas".
B • Conversa	Balões (cliente à esquerda, escritório à direita em navy), quem respondeu em cada balão, respostas rápidas, anexar/gravar áudio, atribuir/transferir/resolver.
C • Contexto do cliente	Processo + fase, próxima parcela, PIX/boleto, status do link — tudo do que já é espelhado. Botões: renovar link, copiar PIX, abrir no ADVBOX.

- **Multi-atendente de verdade:** cada atendente entra com a própria conta (já existem 12). "Pegar para mim", "transferir", "resolver".
- **Identidade real do atendente:** migrar do segredo compartilhado atual para login individual — sem isso não dá para saber quem respondeu.
- **Visual:** respeita 100% o design atual (navy + dourado só em acentos, Cormorant + Lato, sem dark mode, sem barra dourada fixa).

4 · Segurança e LGPD

Como o chat vai carregar conteúdo sensível (documentos, áudios, fotos do cliente), a segurança precisa ser fechada **desde o dia 1** — o oposto do que as tabelas de portal usam hoje.

Risco	Mitigação
Política de acesso permissiva (<code>bot_allow_all</code>) replicada nas tabelas de mensagem	Tabelas novas nascem com acesso negado por padrão ; cliente acessa só via função protegida por token; atendente via login.
Chave de serviço (<code>SUPABASE_SERVICE_ROLE_KEY</code>) não está configurada no Netlify	Configurar antes de fechar a segurança — senão toda escrita do chat para de funcionar.
Token perene na URL (vaza por <code>print/histórico/link</code> encaminhado)	Dar validade ao token, tirar da URL (guardar no aparelho), URLs de mídia curtas (1h) validando a posse.
Mídia gravada por engano no banco	Só referência no banco; binário sempre no bucket privado.
Crescimento infinito do armazenamento	Limpeza automática (retenção) desde o dia 1 — ver custo.

5 · O que se perde tirando o Kommo (e como cobrir)

Atenção: o Kommo **não era só um chat**. Ele roda automações de receita silenciosamente. Aposentá-lo sem reescrever isso faz o escritório perder dinheiro e voltar a ter trabalho manual.

O que o Kommo faz hoje	Risco se sumir	Como cobrir no messenger
Régua de cobrança automática (D+1 / D+7 / D+15)	Cobrança automática para → risco direto de receita	Reescrever para enviar a cobrança como mensagem interna, mantendo a trava anti-duplicação.
Funil de vendas / etapa "ADVBOX"	Perde o acompanhamento do funil	Virar um campo de status na conversa/processo.
NPS + prazo (SLA) das dúvidas	Some o controle de prazo	Migrar para o inbox (cada pergunta vira conversa com prazo).
Notas automáticas no CRM	Perde rastro	Reescrever para gravar evento/nota interna.

Observação

O Kommo já está quebrado hoje (token recusado pela conta), então parte disso já não está rodando. Mas a **régua de cobrança** é a peça mais valiosa — precisa ter destino novo **antes** de aposentar o Kommo de vez.

6 · Custo real (3 cenários)

As bases fixas você já paga hoje: Supabase Pro (~US\$ 25 ≈ R\$ 138/mês) e Netlify (~US\$ 20 ≈ R\$ 110/mês). Os valores abaixo são o que **soma a mais** por causa do messenger (USD 1 ≈ R\$ 5,50).

Volume	Supabase (extra/mês)	Netlify (extra/mês)	Total extra/mês	O que estoura primeiro
100/dia (hoje/meta)	R\$ 0 → ~R\$ 33	R\$ 0	R\$ 0 → ~R\$ 33	Armazenamento de mídia, só após ~4 meses
500/dia (crescimento)	~R\$ 5 → ~R\$ 250	R\$ 0 (modelo de créditos)	~R\$ 5 → ~R\$ 250	Mídia (1º mês) + download começa a contar
1000/dia (escala alta)	~R\$ 140 → ~R\$ 520	R\$ 0 → ~R\$ 55	~R\$ 140 → ~R\$ 575	Armazenamento + download recorrente

Pontos-chave

- **No começo é praticamente de graça.** A 100/dia, o messenger roda dentro do que você já paga por meses.
- **O que cresce sem parar é o armazenamento de mídia.** Cada conversa deposita ~10 MB que não somem sozinhos (~30 GB/mês a 100/dia). Estoura os 100 GB inclusos em ~4 meses; a 500/dia, já no 1º mês.
- **Solução obrigatória desde o dia 1:** retenção (limpeza automática de mídia antiga, ex.: > 6–12 meses) + comprimir foto/áudio antes de subir. Sem isso a conta sobe para sempre.
- **Mídia não passa pela Netlify** (vai direto navegador↔Supabase) — é o que mantém a Netlify barata em qualquer escala.
- **Atenção na Netlify:** a conta pode estar no modelo antigo (teto de 125 mil chamadas/mês por site), que aperta perto de 250–300/dia. Vale migrar para o **modelo de créditos** (cobrar ~1.800/dia) antes de crescer.
- O número de pessoas conectadas ao vivo só chega perto do teto do plano (500) lá pelos 1000/dia — folga, mas é o 3º item a monitorar.

7 · Pré-requisitos (Fase 0 — sem isso nada do resto é seguro)

1. **Configurar `SUPABASE_SERVICE_ROLE_KEY` no Netlify.** Sem ela, ao fechar a segurança das tabelas de mensagem (obrigatório por LGPD), toda escrita do chat para de funcionar.
2. **Liberar microfone/câmera** no arquivo `_headers`, só para a origem do portal.
3. **Criar as tabelas de mensagem com segurança fechada** desde o início (nunca repetir o `bot_allow_all` atual).
4. **Endurecer o token:** dar validade, tirar da URL e validar posse em toda leitura de mídia.
5. **Ligar a limpeza automática de mídia** (retenção) — senão o armazenamento cresce sem teto.

8 · Plano de construção em fases

Fase 0 — Pré-requisitos (~dias)

Chave de serviço + liberar microfone/câmera + criar tabelas seguras + entrar no tempo real. Sem isso nada do resto é seguro.

Fase 1 — MVP só texto maior alavanca (~2-3 semanas)

Chat de texto ao vivo nos dois lados + central de atendimento (inbox 3 painéis, atribuição, respostas rápidas) + push de nova mensagem. **Roda em paralelo ao WhatsApp**, sem aposentar nada. Testar com os 12 clientes ativos.

Fase 2 — Notificação confiável (~1-2 semanas)

Escada push → ao vivo → SMS/e-mail. Onboarding "instale na tela inicial" (obrigatório no iPhone) + "ative os avisos". Só aqui dá para confiar que o cliente vê a mensagem.

Fase 3 — Mídia (~2-3 semanas)

Áudio (gravar + ouvir em 2x), foto, vídeo, documento, miniaturas e limpeza automática.

Fase 4 — Desacoplar o Kommo (~1-2 semanas)

Migrar régua de cobrança, NPS, dúvidas e notas para o messenger, mantendo as travas anti-duplicação.

Fase 5 — Aposentar o WhatsApp (gradual)

Virada por grupos (clientes novos primeiro), com WhatsApp como rede de segurança por mais alguns meses. Nunca corte abrupto.

Total realista: 7 a 11 semanas (inclui 1-2 semanas embutidas de segurança e testes automatizados, obrigatórios porque o deploy vai direto para produção).

9 · Riscos e mitigações

Risco	Mitigação
iPhone não recebe push sem app instalado → cliente não vê a mensagem	Escada de notificação com fallback SMS/e-mail; onboarding de instalação no iPhone.
Aposentar o Kommo sem reescrever a régua de cobrança	Migrar a cobrança automática para o messenger antes de desligar o Kommo.
Infra (tempo real, storage, push) nunca testada sob carga (12 clientes hoje)	Teste de carga + virada por grupos, nunca migrar 100/dia de uma vez.
Deploy direto em produção, sem ambiente de teste	Cobertura de testes automatizados obrigatória em toda função/tela nova (único portão atual).
Armazenamento de mídia cresce para sempre	Política de retenção via tarefa agendada desde o dia 1.
Manter duas tecnologias (portal puro + painel React)	Reaproveitar helpers e padrões existentes; aceitar implementar o chat duas vezes (cliente e escritório).

10 · Recomendação final

Vale a pena — com cabeça fria sobre o cronograma e sobre o WhatsApp. O messenger é a evolução natural do que você já construiu; a fundação está pronta e ociosa, o portal mal foi usado (janela ideal), e o custo é baixíssimo no início e controlável em escala (desde que a limpeza de mídia entre no dia 1).

Por onde começar

1. **Fase 0 + Fase 1** (chat só-texto ao vivo, rodando ao lado do WhatsApp). Em ~3 semanas você já tem um chat real, com central multi-atendente, testável com clientes de verdade — e nada quebra se der errado.
2. **Decidir cedo sobre o SMS/e-mail.** Sem essa rede de segurança, o objetivo "só por aqui" não se sustenta no iPhone.
3. **Não desligar o WhatsApp nem o Kommo** até a régua de cobrança estar reescrita e a escada de notificação provada. Virar por grupos, não de uma vez.

Anexo A · Referências técnicas

Endpoints novos (Netlify Functions)

messenger-abrir · messenger-listar · messenger-send · messenger-marcar-lido · messenger-upload-url · messenger-midia-url · messenger-notify · messenger-fallback (SMS/e-mail, via tarefa agendada).

Arquivos-chave para a construção

Arquivo	Papel no messenger
client/portal.html	Onde entra a aba "Mensagens" (cliente). Carregar o cliente de tempo real só nessa aba.
client/src/components/PortalClientePanel.jsx (+ novo CentralAtendimento.jsx)	Central de atendimento (escritório).
client/public/_headers	Bloqueador de microfone/câmera a corrigir.
client/netlify/functions/_lib/botDb.mjs	Acesso ao banco / assinatura de URLs (precisa da chave de serviço).
client/netlify/functions/portal-push.mjs · client/public/portal-sw.js	Notificação push já pronta — reaproveitar.
client/netlify/functions/cobranca-regua.mjs	Régua de cobrança a migrar do Kommo.

Premissas de volume (cenário 100 atendimentos/dia)

~12 mensagens por conversa · ~35% das mensagens com mídia · ~10 MB de mídia por conversa · ~30 GB/mês de mídia acumulando · pico de 30–60 conexões ao vivo simultâneas · < 5 mensagens/segundo no tempo real (folga enorme no plano Pro).

Documento gerado em 14/06/2026 a partir de análise multiagente do código-fonte do projeto cbc-contratos, da arquitetura proposta e da modelagem de custo (preços oficiais Supabase Pro e Netlify, junho/2026). Valores em R\$ são estimativas com USD 1 ≈ R\$ 5,50 e podem variar com câmbio e uso real. Documento interno — CBC Advogados.